

Week 02 Workshop

EXERCISE:

Workshop 2 - Rust Train-ing

PRE-WORKSHOP PREPARATION:

There are three things which may help prepare for this workshop:

- Complete the "Data analysis exercise".
- Watch the second week's lectures.
- Download and read through the [starter code](#).

This is not required, but will be very useful!

If you'd like to feel extra comfortable during the workshop, this pre-workshop work is available:

- Fetch the [starter code](#).
- Read the data from `trains.csv` into an appropriate data structure (extra code for this has been provided, you will need to copy that code in and fix any bugs).
- For each year of data, find the most and least used stations.
- For each 10th of a degree of latitude and longitude, find the most and least used stations.
- Which station has had it's usage increase the most over the last years?

IN-WORKSHOP REVISION:

Your tutor will quickly remind you of:

- the Vec and HashMap types.
- Simple derive macros (Copy, Clone, Debug).
- The purpose of the `serde` crate (with reference to the starter code).

If you have any questions, now is a great time to ask them!

In this workshop, we will be using Rust to play with some data about NSW Trains!

The purpose of this workshop is to:

- Further practice basic rust features and error handling.
- Experiment with Rust's many collection types.
- Have fun!

This tutorial is **not** designed to require any data analysis experience; and it is not our goal to teach this to you; however it will assume simple mathematical concepts and some creativity!

In the workshop, you will be asked to get into pair-programming groups of 2 people. Only one person should be coding work on different problems. If someone has already done the pre-workshop work, start with that code. If nobody has, get the starter code [from this tar file](#). If more than two people have done the work, commence a duel to the death (or, discuss which code is better for which tasks, and use the most appropriate code/a combination of the two).

Teams can pick tasks to work on from the list below. Once a task is done, they should call over a tutor to have a chat about the task. Tutors will also be happy to provide help on rust issues (both related to getting code working, and design). Teams can "choose their own adventure", but we encourage teams with lots of experience to pick harder tasks as a group. Teams should also use both functional and imperative approaches, so we can compare the two. **There are more tasks than can reasonably be completed in a day!**

The Tasks

- Use the provided starter code (and the useful code in a separate file) to represent the CSV data in a useful way. The extra code you have been provided should be copied into your main function (you may need to fix bugs that arise when you copy it across).
- Find the most and least used stations on the NSW network at each time of day.
- Allow a user to search for a station, and show it's busiest times of day, and busiest year.
- Which station has had it's yearly utilisation increase the most in the last 5 years?
- Which station had the biggest percentage change in use over 2020?

- What is the north-most, south-most, east-most and west-most station?
- Find the two closest stations and the two furthest away from each other.
- Sort stations by their distance from central, and by their usage. Is the order similar, or different? Are there outliers?
- (hard) A meteor is headed for sydney! It is headed for a train station, but we don't know which one. It will destroy all stations within 2 kilometers of it. For each 4-hour period, make a list of which stations would be best or worst to hit.
- (very hard) Make a graph of an interesting result found above, and post it to [/r/dataisbeautiful](#).

At the end, we'll come back together to review interesting bugs/errors, and interesting ways of completing the tasks. We'll also discuss functional and imperative approaches.

COMP6991 23T1: Solving Modern Programming Problems with Rust is brought to you by
the [School of Computer Science and Engineering](#)
at the [University of New South Wales](#), Sydney.

For all enquiries, please email the class account at cs6991@cse.unsw.edu.au

CRICOS Provider 00098G

[Login as tutor](#)